ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Разработка REST API для отслеживания курсов валют (FastAPI + SQLAlchemy)

---

1. Цель работы

Разработать асинхронное веб-приложение на FastAPI с использованием ORM SQLAlchemy для взаимодействия с базой данных SQLite. Приложение предоставляет функционал регистрации пользователей, подписки на отслеживание актуальных курсов валют с автоматическим получением данных от Центрального банка РФ.

---

2. Реализованная структура проекта

```
ЛР3/
├── alembic/                # Миграции базы данных (Alembic)
│   ├── versions/           # Файлы версионирования схемы БД
│   ├── env.py              # Конфигурация среды миграций
│   └── script.py.mako      # Шаблон генерации новых миграций
├── app/                    # Основной код приложения
│   ├── routers/            # Логическое разделение эндпоинтов
│   ├── database.py         # Настройка асинхронного подключения
(AsyncSession)
│   ├── main.py             # Точка входа, инициализация FastAPI и
зависимостей
│   ├── models.py           # SQLAlchemy ORM модели (User, Currency,
Subscription)
│   └── schemas.py          # Pydantic модели для валидации запросов/ответов
├── services/               # Внешние сервисы и бизнес-логика
│   └── cbrf.py             # Асинхронный парсер XML API ЦБ РФ
├── static/                 # Статические ресурсы (фронтенд)
│   ├── css/                # Стили (Bootstrap, кастомные стили)
│   ├── images/             # Изображения и иконки
│   └── js/                 # Клиентская логика (Fetch API, динамические
таблицы)
├── templates/              # Jinja2 HTML-шаблоны страниц
├── alembic.ini             # Конфигурационный файл Alembic
```

```
├── requirements.txt          # Список зависимостей проекта
└── README.md                 # Инструкция по запуску и описание
```

---

3. Модели данных (SQLAlchemy ORM)

3.1. Модель пользователя (UserBase)

```
class UserBase(Base):
    __tablename__ = "user"
    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True,
nullable=False)
    email: Mapped[str] = mapped_column(String(100), unique=True,
nullable=False)
    created_at: Mapped[str] = mapped_column(String(50), nullable=False)

    # Связь с подписками (один-ко-многим)
    subscriptions: Mapped[list["Subscription"]] = relationship(
        "Subscription", back_populates="user"
    )
```

3.2. Модель валюты (Currency)

```
class Currency(Base):
    __tablename__ = "currency"
    id: Mapped[int] = mapped_column(primary_key=True)
    code: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
    name: Mapped[str] = mapped_column(String(100), nullable=False)

    # Связь с подписками
    subscriptions: Mapped[list["Subscription"]] = relationship(
        "Subscription", back_populates="currency"
    )
```

3.3. Модель подписки (Subscription)

```
class Subscription(Base):
    __tablename__ = "subscription"
    user_id: Mapped[int] = mapped_column(ForeignKey("user.id"),
primary_key=True)
    currency_id: Mapped[int] = mapped_column(ForeignKey("currency.id"),
```

```
primary_key=True)

    # Составной первичный ключ
    __table_args__ = (PrimaryKeyConstraint('user_id', 'currency_id'),)

    # Связи с пользователями и валютами
    currency: Mapped["Currency"] = relationship("Currency",
back_populates="subscriptions")
    user: Mapped["UserBase"] = relationship("UserBase",
back_populates="subscriptions")
```

---

4. Реализованные эндпоинты API

4.1. Модуль пользователей (`/users`)

Метод	Эндпоинт	Описание	Статус-коды
POST	`/users`	Создание нового пользователя	200 (успех), 409 (дубликат)
GET	`/users`	Получение списка всех пользователей	200
GET	`/users/{user_id}`	Получение информации о пользователе + его подписки	200, 404
DELETE	`/users/{user_id}`	Удаление пользователя	200, 404

Пример запроса на создание:

```
POST /users
Content-Type: application/x-www-form-urlencoded

username=ivan_ivanov&email=ivan@example.com
```

Пример ответа:

```
{
  "id": 1,
  "username": "ivan_ivanov",
  "email": "ivan@example.com",
  "created_at": "2026-04-15 10:30:00"
}
```

4.2. Модуль валют (`/currencies`)

Метод	Эндпоинт	Описание	Статус-коды
GET	`/currencies`	Получение списка всех валют из БД	200
POST	`/currencies/update`	Обновление курсов валют с API ЦБ РФ	200, 500
GET	`/api/currencies`	API endpoint для JSON (используется в JS)	200

Особенности реализации `/currencies/update`:

- Асинхронный GET-запрос к `https://www.cbr.ru/scripts/XML_daily.asp` через `httpx`
- Парсинг XML с использованием `xml.etree.ElementTree`
- Извлечение данных: `CharCode`, `Name`, `Value`, `Nominal`, `Date`
- Обновление существующих валют и добавление новых через `INSERT OR REPLACE`
- Возвращается количество обновлённых валют

Пример ответа:

```
{
  "status": "success",
  "message": "Обновлено 34 валют",
  "count": 34
}
```

4.3. Модуль подписок (интегрирован в `/users`)

Подписки реализованы через страницу пользователя `/users/{user_id}`, где отображается таблица валют, на которые подписан пользователь.

Функциональность:

- Просмотр подписок пользователя с данными о валютах (код, название, курс)
- Использование `JOIN` через `relationship` для получения данных о валютах
- Оптимизация запросов через `joinedload()` для избежания N+1 проблемы

---

5. Технические особенности реализации

5.1. Асинхронность

- Использован `sqlalchemy.ext.asyncio` для асинхронных сессий
- Все эндпоинты объявлены как `async def`
- Применён `aiosqlite` — асинхронный драйвер для SQLite
- Внешние запросы к ЦБ РФ выполняются через `httpx.AsyncClient`

5.2. Dependency Injection

```
"""Зависимость для получения сессии БД"""
    async with async_session_factory() as session:
        try:
            yield session
        except Exception:
            # Если в коде эндпоинта произошла ошибка — откатываем изменения
            await session.rollback()
            raise
        # Сессия закроется автоматически благодаря конструкции 'async with'
```

5.3. Валидация данных

- Pydantic модели для схем запросов/ответов
- Проверка уникальности `username` и `email`
- Обработка `IntegrityError` при попытке создания дубликатов
- Возврат корректных HTTP статус-кодов:
  - `404 Not Found` — ресурс не найден
  - `409 Conflict` — конфликт данных (дубликат)
  - `500 Internal Server Error` — ошибка сервера

5.4. Работа с шаблонами

- Jinja2 для рендеринга HTML-страниц
- Bootstrap 5 для адаптивного дизайна
- Динамическое обновление таблиц через JavaScript (Fetch API)
- Обработка ошибок на клиенте с информативными сообщениями

5.5. Миграции БД

- Alembic для управления миграциями
- Автоматическая генерация миграций через `alembic revision --autogenerate`

- Применение миграций при старте сервера (опционально)

---

6. Взаимодействие с API ЦБ РФ

Алгоритм обновления курсов:

1. **GET-запрос** к `https://www.cbr.ru/scripts/XML_daily.asp`
2. **Парсинг XML:**

```
import xml.etree.ElementTree as ET

root = ET.fromstring(xml_content)
for valute in root.findall('Valute'):
    char_code = valute.find('CharCode').text
    name = valute.find('Name').text
    value = float(valute.find('Value').text.replace(',', '.'))
    nominal = int(valute.find('Nominal').text)
```

3. **Обновление БД:**
   - Проверка существования валюты по `code`
   - `UPDATE` существующей записи или `INSERT` новой
4. **Возврат результата** с количеством обновлённых записей

---

7. Примеры использования

7.1. Создание пользователя и подписка на валюту

```
# 1. Создать пользователя
curl -X POST "http://localhost:8000/users" \
  -d "username=alex&email=alex@test.ru"

# 2. Обновить курсы валют
curl -X POST "http://localhost:8000/currencies/update"

# 3. Посмотреть пользователя с подписками
curl "http://localhost:8000/users/1"
```

7.2. Работа через веб-интерфейс

1. Открыть `http://localhost:8000/users`

2. Нажать "Создать пользователя", заполнить форму
3. Перейти на страницу пользователя `/users/{id}`
4. Нажать "Обновить валюты" для получения актуальных курсов
5. Выбрать валюту из списка для подписки

---

8. Зависимости проекта (requirements.txt)

```
fastapi>=0.135.0
uvicorn>=0.44.0
sqlalchemy>=2.0.49
alembic>=1.18.0
aiosqlite>=0.20.0
httpx>=0.28.0
pydantic>=2.13.0
jinja2>=3.1.0
python-multipart>=0.0.6
```

---

9. Инструкция по запуску

```
# 1. Клонировать репозиторий
git clone https://github.com/huksleva/proga-4
cd proga-4/ЛР3

# 2. Создать виртуальное окружение
python -m venv venv
venv\Scripts\activate  # Windows
source venv/bin/activate  # Linux/Mac

# 3. Установить зависимости
pip install -r requirements.txt

# 4. Применить миграции
alembic upgrade head

# 5. Запустить сервер
uvicorn app.main:app --reload

# 6. Открыть в браузере
```

```
# http://localhost:8000/users
# http://localhost:8000/docs (Swagger документация)
```

---

10. Реализованный функционал (чек-лист)

Работа с пользователями:

- [x] ~~Создание пользователя (POST /users)~~
- [x] ~~Получение списка пользователей (GET /users)~~
- [x] ~~Получение информации о пользователе (GET /users/{id})~~
- [x] ~~Удаление пользователя (DELETE /users/{id})~~
- [x] ~~Обновление информации (опционально)~~

Работа с валютами:

- [x] ~~Получение списка валют из БД (GET /currencies)~~
- [x] ~~Обновление курсов с API ЦБ РФ (POST /currencies/update)~~
- [x] ~~Парсинг XML ответа~~
- [x] ~~Сохранение актуальных курсов~~

Работа с подписками:

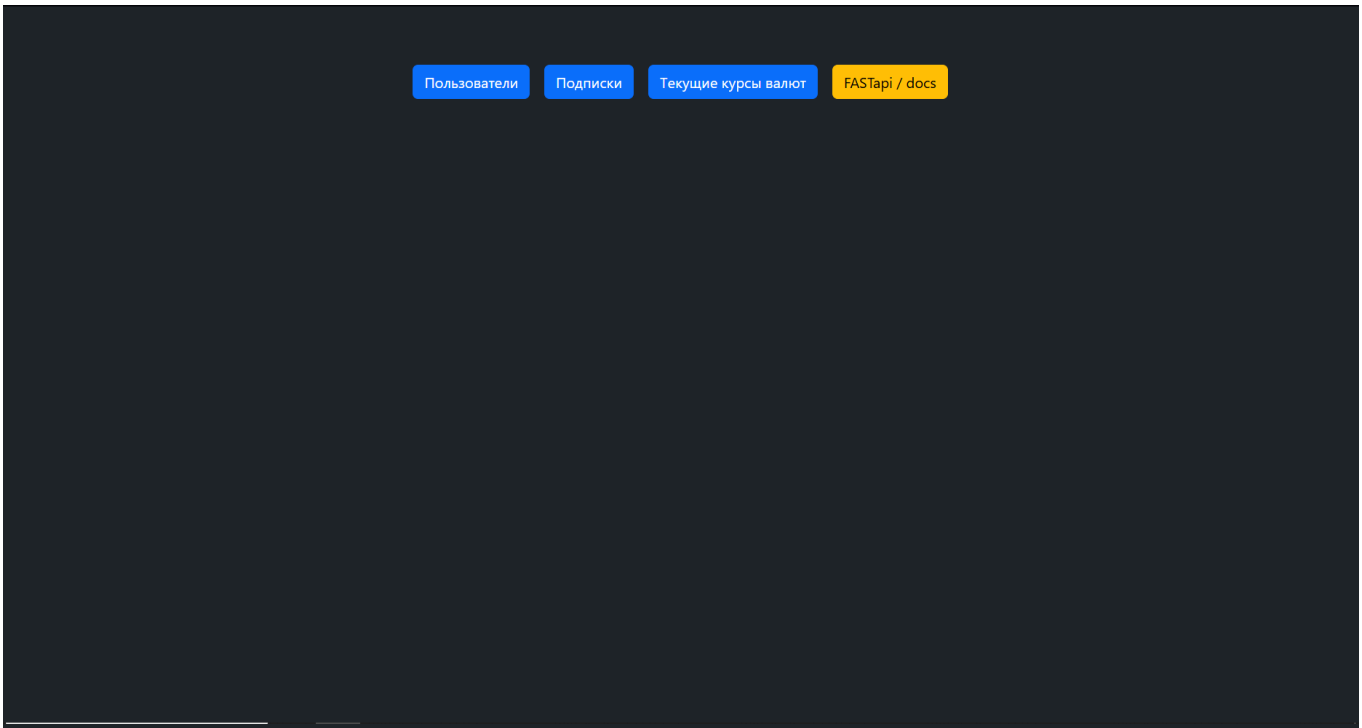
- [x] ~~Просмотр подписок пользователя~~
- [x] ~~Связь Many-to-Many через таблицу Subscription~~
- [x] ~~Отображение данных о валютах в подписках~~

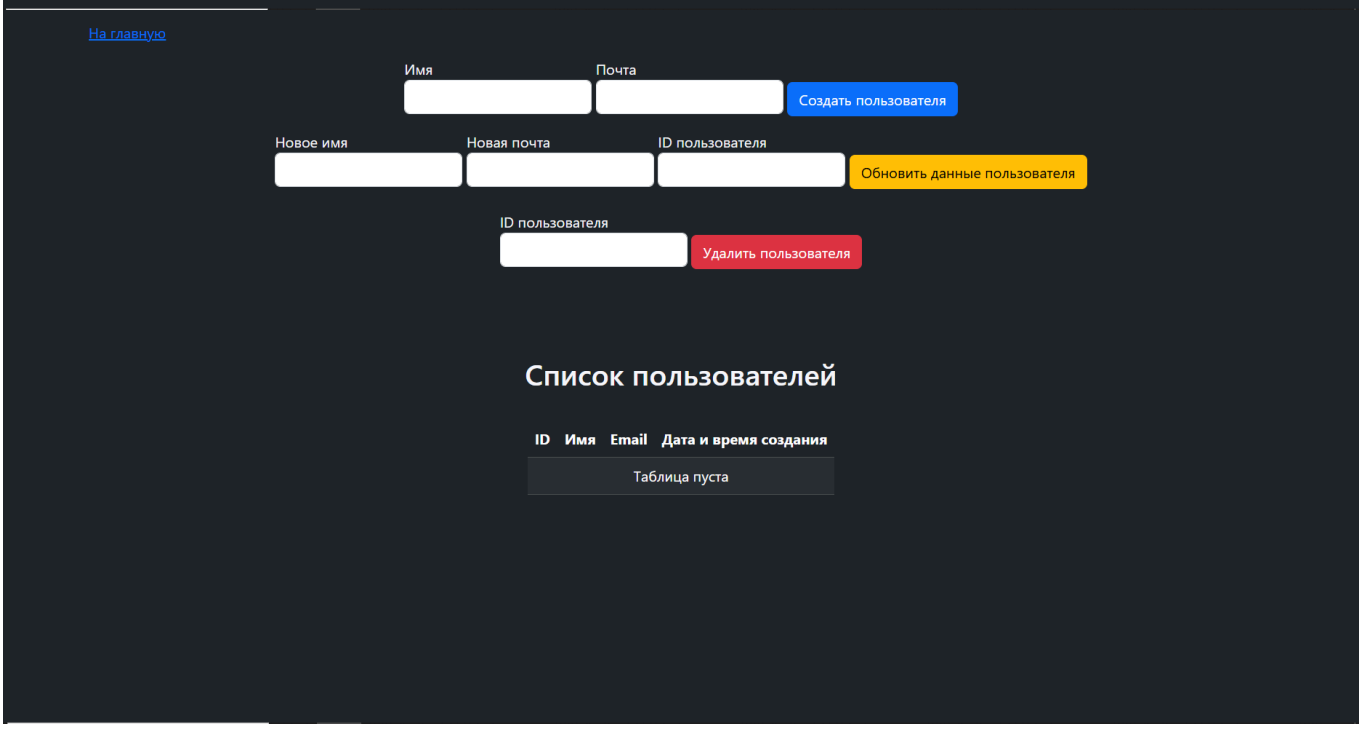
Технические требования:

- [x] ~~Асинхронный код (async/await)~~
- [x] ~~SQLAlchemy ORM с декларативным стилем~~
- [x] ~~Асинхронные сессии (AsyncSession)~~
- [x] ~~Dependency Injection~~
- [x] ~~Pydantic модели~~
- [x] ~~Обработка ошибок и валидация~~
- [x] ~~Корректные HTTP статус-коды~~
- [x] ~~Миграции Alembic~~
- [x] ~~Веб-интерфейс (HTML + JS)~~

---

13. Скриншоты работы приложения





На главную

ID пользователя

ID валюты

Добавить подписку

ID пользователя

ID валюты

Удалить подписку

Подписки пользователей

User ID	Currency ID
Нет активных подписок	

На главную Пользователи

Информация о пользователе

ID	Имя	Email	Дата и время создания
1	Leonid	leonid005xc@gmail.com	15:04:2026 01:43

Информация о подписках пользователя

ID	Код	Название
16	USD	Доллар США
39	TJS	Сомони

На главную

Обновить данные о курсах валют

Текущие курсы валют

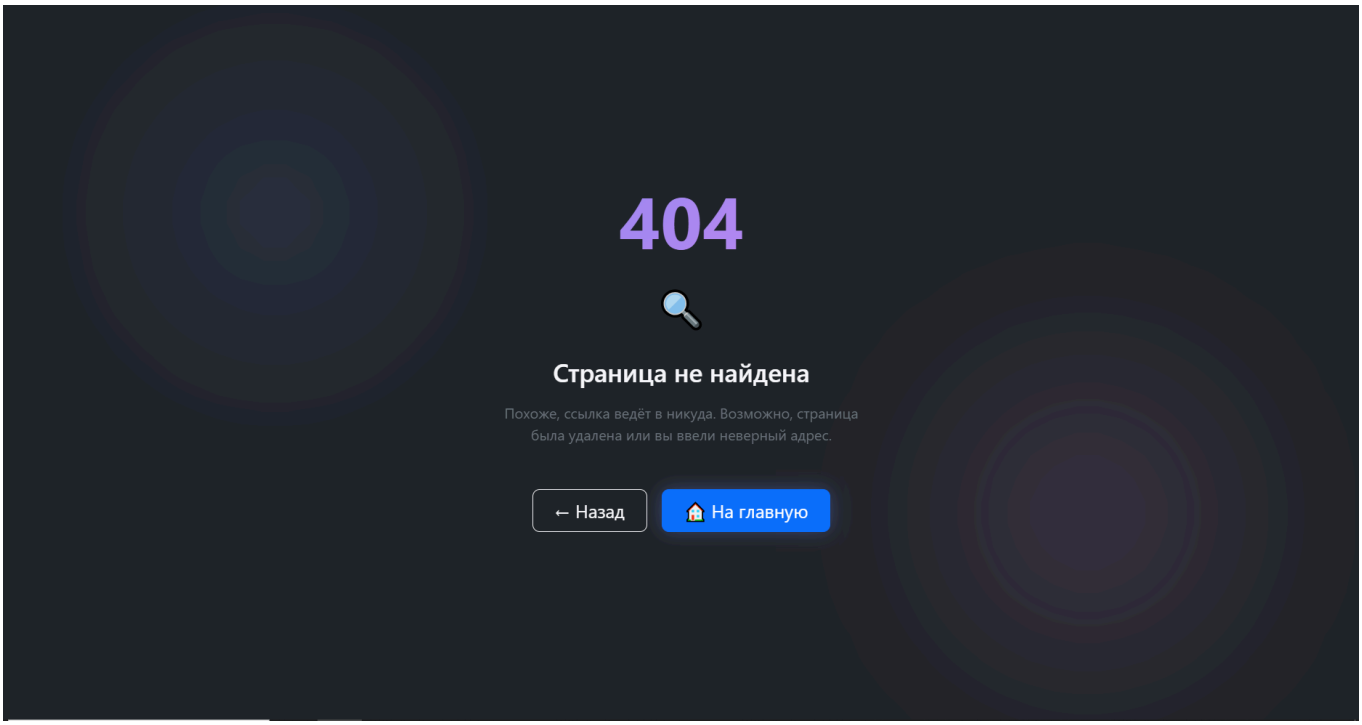
ID	Код	Название
1	AUD	Австралийский доллар
2	AZN	Азербайджанский манат
3	DZD	Алжирских динаров
4	GBP	Фунт стерлингов
5	AMD	Армянских драмов
6	BHD	Бахрейнский динар
7	BYN	Белорусский рубль
8	BOB	Боливиано
9	BRL	Бразильский реал
10	HUF	Форинтов
11	VND	Донгов
12	HKD	Гонконгских долларов

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

- GET / Main Page
- GET /users Users Page
- POST /users Users Page
- GET /currencies Currencies Page
- GET /api/currencies/ Get Currencies
- GET /subscriptions Subscriptions Page
- POST /subscriptions Subscriptions Page
- DELETE /subscriptions Delete Subscription
- GET /users/{user_id} Users Page
- DELETE /users/{user_id} Delete User
- PUT /users/{user_id} Update User Info
- POST /currencies/update Update Currencies Page



---

12. Выводы

В ходе выполнения лабораторной работы было разработано полнофункциональное REST API приложение с использованием современных технологий:

1. **FastAPI** обеспечил высокую производительность и автоматическую документацию
2. **SQLAlchemy ORM** упростил работу с базой данных, позволив использовать декларативный стиль
3. **Асинхронность** (asyncio, aiosqlite, httpx) обеспечила эффективную обработку запросов
4. **Alembic** предоставил удобное управление миграциями схемы БД
5. **Jinja2 + Bootstrap** создали удобный веб-интерфейс для взаимодействия с API